SME CAPITAL FUNDING OPTIMIZATION · ARTIFACT 1 OF 4

# Data Cleaning Walkthrough

How a messy export of 1,186 funding applications was diagnosed, repaired and turned into an analysis-ready dataset in Python — without discarding a single business for gaps the form itself created.

**Prepared by** Ndivhuwo

**Tooling** Python · pandas · NumPy · openpyxl

**Input** capital_matching_applications.csv (1,186 × 27) **Output** Capital_Matching_Cleaned_Data.xlsx (1,116 × 41)

The JSE's SME Capital Matching form exported clean-looking headers hiding genuinely messy values. Before any scoring was possible, the data had to be made trustworthy. This document walks through what arrived broken, why each problem would have corrupted the analysis, and how it was resolved — the reasoning a reviewer needs to judge the work, not just the result.

## 01 The starting position

The raw file held **1,186 rows across 27 self-reported fields**, one row per application submitted through a public web form. Nothing came from a registry or a credit bureau; the analysis had to work with exactly what applicants typed. A first inspection showed that roughly **one field in ten arrived unusable as captured** — enough that scoring on the raw data would have been meaningless.

> **GUIDING PRINCIPLE**
>
> Repair, don't reject. Many gaps were created by the form, not by the applicant. Discarding a business for a blank the form failed to enforce would silently bias the pipeline against otherwise strong SMEs, so the cleaning was built to **recover** data wherever possible and to **neutralise** — never penalise — what genuinely could not be recovered.

## 02 The problems, and how each was resolved

Every defect below was found by profiling the columns — checking value types, unique counts, ranges and null rates — before writing a single fix. Each fix was then applied as an explicit, documented transformation so the whole process re-runs identically on the next intake.

| The problem | Why it breaks the analysis | How it was resolved in Python |
|---|---|---|
| **Underscore-padded money**<br>e.g. `250000_____` | A funding amount stored as text with trailing junk can't be summed, compared or ratio'd. Left as-is, almost every amount reads as missing. | Stripped the padding and any stray non-numeric characters, then parsed the remainder into a clean integer Rand value. |

| The problem | Why it breaks the analysis | How it was resolved in Python |
|---|---|---|
| **Revenue as text bands**<br>e.g. "R1m – R5m" | Revenue drives the single heaviest scoring pillar. As free text it is neither orderable nor comparable, so growth and size can't be measured. | Mapped each band onto an **ordered 0–6 scale** and attached a representative Rand midpoint, preserving both rank and magnitude for later calculation. |
| **Implausible outliers** | A handful of amounts ran into the billions — clearly capture errors that would dominate every total and distort ratios. | Applied a **R2 billion cap**: any value above it is treated as a capture error rather than a real figure, protecting the aggregates. |
| **Inconsistent provinces**<br>"KZN", "Kwazulu Natal"... | Nine provinces spelled several ways each fragment every province-level count and every map in the dashboard. | Standardised all spelling variants to a single canonical name per province (e.g. `KZN` → `KwaZulu-Natal`); tidied city/town text alongside. |
| **Ownership as bands** | Black / women / youth / disability ownership are needed as numbers for the transformation pillar, but arrived as categorical ranges. | Converted each ownership band to a midpoint percentage, and extracted the numeric **B-BBEE level** from its label. |
| **Free text in numeric fields & malformed IDs** | Notes sitting in headcount or amount columns, and malformed registration numbers, silently poison any numeric operation. | Coerced numeric columns to numbers (isolating non-numeric entries), and flagged malformed registration numbers rather than dropping the business. |
| **Duplicate submissions** | The same business counted twice inflates every total and can double-rank an applicant. | Two de-duplication passes on a normalised key (emails lower-cased first): **68 exact copies** and **2 re-submissions** removed — 70 rows in all. |
| **Blanks in required fields** | Investors need fields the form didn't force. Dropping every incomplete row would delete real, viable businesses. | Kept the businesses; missing answers are carried forward to be scored **neutrally** at the modelling stage (see Artifact 3), not zeroed. |

## 03 The recovery, in numbers

The clearest measure of the cleaning is how much usable data it rescued from columns that were effectively empty in the raw file.

**10 → 1,109**
USABLE FUNDING AMOUNTS

**0 → 1,112**
USABLE 2025 REVENUE FIGURES

**70**
DUPLICATE ROWS REMOVED

**1,116**
UNIQUE BUSINESSES RETAINED

Funding amounts went from **10 usable values to 1,109**, and 2025 revenue from **none to 1,112** — the two fields the entire scoring model leans on hardest. The 1,186 raw rows resolved to **1,116 unique businesses**, none discarded for a gap it did not create.

## 04 What the step produced

The header renaming was retained as a **data dictionary**, so every tidy column traces back to its original survey question. The result is `Capital_Matching_Cleaned_Data.xlsx` — **1,116 rows × 41 columns**, every financial field a real number, every category standardised, every business accounted for. This is the clean foundation the scoring model (Artifact 3) is built on.

> **WHY IT'S REPRODUCIBLE**
>
> The whole workflow lives in a single documented notebook (`Data_Cleaning.ipynb`) where every code cell is preceded by a plain-language explanation. Re-running it on a fresh intake produces the same clean output by the same rules — cleaning is a repeatable pipeline here, not a one-off manual scrub.


Ndivhuwo · ndivhuwojse@gmail.com · github.com/MainDevWork